Stage 3 Presentation - Slide-by-Slide Outline Abhinav Bachu, Hanshul Bahl, Alejandra Arias CS 498 / Data Management in the Cloud 10-minute talk, ~12 slides

This document tells you exactly what to put on each slide and what to say while it is up. The four required content points from the assignment (data model overview, 2+ query demos, critique, lessons learned) are spread across slides 3, 5-10, 11, and 12 respectively. We implemented all 6 queries (the assignment minimum for a 3-person team is 4 of 6, so this earns the +10 extra-credit) and loaded the full InsideAirbnb datasets for all 4 cities (the assignment's "use all or nearly all the data" worth 10 percentage points). Total target speaking time: 9-10 minutes; ~45 seconds per slide except for demos which run a bit longer.

Slide 1 - Title

On the slide:

- Title: "MongoDB + AirBnB - Stage 3 Prototype"
- Course: CS 498 / Data Management in the Cloud
- Team: Abhinav Bachu, Hanshul Bahl, Alejandra Arias
- Date

Speaker notes (~30 sec): Quick intro. "We're going to walk through the prototype we built on top of our Stage 2 design - a working MongoDB + AirBnB system that loads the full InsideAirbnb datasets for four cities and runs all six queries from the question document end-to-end."

Slide 2 - Project Recap and Stage 3 Scope

On the slide:

- One-line summary of Stage 1 (system + dataset selection: MongoDB + InsideAirbnb 4 cities)
- One-line summary of Stage 2 (4 collections, 8 indexes, pseudocode for all 6 queries)
- Stage 3 scope (3 short bullets):
  - Loader against the full real datasets (no row caps)
  - **All 6 of 6** queries implemented and benchmarked (assignment minimum is 4 for a 3-person team)
  - `explain('executionStats')` captured for every query
- "Two ways to run it: `npm start` (CLI evidence pipeline) or `npm run web` (interactive UI)"

Speaker notes (~45 sec): Frame the work. "Stage 1 picked the system and dataset. Stage 2 was design on paper. Stage 3 is the prototype - actually loading data, actually running queries, and proving the indexes we designed are the indexes the planner uses. We hit both of the assignment's bonus targets: all 6 queries (instead of the required 4) and full data load (instead of a subset)."

Slide 3 - Data Model Overview

On the slide:

- Diagram or table of the 4 collections:

    - `listings` (per-listing metadata)
    - `calendar` (per-day availability/pricing)
    - `reviews` (per-review records)
    - `neighborhoods` (canonical neighborhood names)
- Index list (small font, 8 entries):
    - `listings { city, listing_id }` unique
    - `listings { city, neighborhood, room_type }`
    - `listings { host_id, city }`
    - `calendar { city, listing_id, date }`
    - `calendar { city, date, available }`
    - `reviews { city, listing_id, date }`
    - `reviews { reviewer_id, listing_id, date }`
    - `neighborhoods { city, neighborhood }` unique
- Caption: "All 8 indexes carried over from Stage 2 unchanged."

Speaker notes (~45 sec): Walk the audience through the data model in 3 sentences. "Four collections, each one maps to one source CSV file from InsideAirbnb. The high-volume time-series collections - calendar and reviews - stay separate from listings so we don't have giant array rewrites on every booking change. Indexes are tuned per query."

Slide 4 - Loader Pipeline + Data Coverage

On the slide:

- Simple horizontal flow diagram:
    - InsideAirbnb URLs -> fetch -> gunzip stream -> csv-parse -> batch bulk insert -> create indexes
- "Streaming + batched inserts so the full multi-million-row load fits in heap"
- **Counts loaded in the full-data run** (recorded — `out/load_evidence.txt`):
    - listings: **63,482** (LA 45.6k · Portland 4.5k · Salem 280 · San Diego 13.2k)
    - reviews: **3,200,722**
    - neighborhoods: **481**
    - calendar: **23,170,957** (the full 365-day 2025-12 snapshot for every listing)
- Caption: "**This is the entire InsideAirbnb dataset for all 4 cities**, not a subset. (Assignment: 'use of all or nearly all the data is worth 10 percentage points'.) Total load + index build: ~6.5 minutes."

Speaker notes (~1 min): "We stream the gzipped CSVs from InsideAirbnb directly through gunzip and csv-parse so we never hold a full file in memory. Inserts are batched at 2-10k rows per call so even the 1.5GB Los Angeles reviews file loads cleanly. We removed the row caps for the recorded run - this is the full real-world data, every listing, every calendar day, every review across all four cities. The same code would run unmodified against a real Atlas cluster - we just swap the URI."

Slide 5 - Demo: Query 1 (Portland 2-day Search)

On the slide:

- Query name and goal: "Listings available on both days of a requested 2-day window in Portland, sorted by rating"
- The pipeline (small code block, abbreviated):

```
$match { city, date IN [d1,d2], available }
$group by listing_id, count = 2
$lookup listings -> $sort review_scores_rating
```

- Sample result row from the recorded run (from `out/q1_results.json`)
- Execution stats from `explain()` (full-data run):
  - **IXSCAN on `city_1_date_1_available_1`**
  - 3,012 keys examined / 3,012 docs examined / 1,826 returned pre-limit / **112 ms** over the 1.6M-row Portland calendar partition

Speaker notes (~1 min): Show the query, then show that it actually hits the index. "We pick the demo dates dynamically from the loaded data so this always returns something live. The execution plan confirms we get a clean index scan. No collection scan, no sort spill."

Slide 6 - Demo: Query 2 (Empty Neighborhoods in a Month)

On the slide:

- Query name and goal: "For a given city and month, which neighborhoods had zero listings available?"
- Pipeline diagram:
  1. `$match` calendar by city + date range + available -> `$group` by listing_id
  2. `$lookup` listings -> project neighborhood -> `$group` -> active set
  3. Anti-join in app code against full `neighborhoods` for that city
- Sample result excerpt: "**2 of 95** Portland neighborhoods were dark in 2026-06" (from `out/q2_results.json`)
- "Indexes hit: `calendar { city, date, available }`, `listings { city, listing_id }`, `neighborhoods { city, neighborhood }`"

Speaker notes (~45 sec): "This one is interesting because it's an *anti*-query - we want what *isn't* there. We compute the active set with one indexed aggregation, then diff against the canonical neighborhood list."

Slide 7 - Demo: Query 3 (Salem Entire-Home Availability Windows)

On the slide:

- Query name and goal: "For each Salem entire-home listing, find every contiguous bookable window in a given month, respecting per-day `minimum_nights`"
- Algorithm (3 bullets):
  1. One indexed `find` per Salem entire-home listing (via `listings { city, neighborhood, room_type }`)
  2. One sorted `find` over `calendar { city, listing_id, date }` for the month
  3. App-side: walk consecutive `available:true` runs; keep day d only if remaining-run-length >= min_nights[d]
- Sample card: "Listing X -> 2025-12-04 -> 2025-12-09 (5 nights, min 2)"

Speaker notes (~1 min): "Lifestyle constraint: a listing might say 'available' for a Saturday but require a 7-night minimum. If only Saturday and Sunday are open you can't actually book it. We model this faithfully: scan available runs and exclude days whose remaining run is shorter than that day's `minimum_nights`."

Slide 8 - Demo: Query 4 (Portland Booking Trend Mar-Aug)

On the slide:

- Query name and goal: "Total bookable nights per month for Portland entire-home listings, March through August"
- "Reuses Q3's interval logic in a 6-iteration loop"
- Bar chart (one bar per month) with total bookable nights
- Recorded numbers (Portland entire-home, 2026): Mar **50,547** · Apr **48,455** · May **50,928** · Jun **43,364** · Jul **45,930** · Aug **46,143** total bookable nights

Speaker notes (~45 sec): "Q4 is Q3 in a loop. We deliberately did *not* turn this into one giant aggregation - the per-month numbers are each computed from one indexed scan plus the same interval scanner. The structure makes it easy to add more months or filter by neighborhood later."

Slide 9 - Demo: Query 5 (December Reviews per City per Year)

On the slide:

- Query name and goal: "Count December reviews grouped by city and year"
- The pipeline (small code block):

```
$match date matches "____-12-__"
$group by { city, year: $substr(date,0,4) }, sum
$sort city, year
```

- Sample result rows (3-4 rows from `out/q5_results.json` — 62 (city, year) groups returned)

- Execution note: "Full-collection aggregation by design — no useful index prefix. **COLLSCAN over 3,200,722 reviews → 193,340 December matches in 1,775 ms.** If reviews grew to 100M+, add `reviews { date: 1 }`."

Speaker notes (~45 sec): Highlight the date-as-string design choice paying off. "We stored dates as strings. December extraction is a one-character substring match on a regex - no `$toDate` cost. Stage 2 reasoned about this in terms of storage, but the bigger win turned out to be query simplicity."

Slide 10 - Demo: Query 6 (Re-book Reminder + Same-Host Listings)

On the slide:

- Query name and goal: "For repeat reviewer-listing pairs, was the listing available in the same month as their previous review? What other listings does that host have in the same city?"
- Multi-step diagram (3 boxes):
  1. Aggregate `reviews` -> repeat (city, listing, reviewer) groups
  2. For each: `findOne` calendar (available?) + `findOne` listing details + `find` other host listings
  3. Emit enriched record
- Sample output excerpt (1 record, 3-4 fields shown)
- "All step-2 reads hit indexes designed in Stage 2 - **no new indexes needed for any of the 6 queries**."

Speaker notes (~1 min): "Most interesting query because it touches all 4 collections. We could have forced this into a single aggregation with nested `$lookup` s, but breaking it into one aggregate plus a few indexed point reads per group is much easier to read and debug. Importantly, every per-record read in step 2 lands on an index we already designed for some other query in Stage 2 - that's a strong signal the index design generalizes."

Slide 11 - Critique: What We'd Change

On the slide: (3 bullets)

- **Date-as-string call was right** but for query simplicity, not storage. Update the design rationale.
- **`host_id_1_city_1` index has wrong key order** - should be `{ city, host_id }` so it shares a `city` prefix with our other 3 listings indexes.
- **Neighborhoods could be embedded** as a per-city array - eliminates the `$lookup` in Q2 entirely.

Speaker notes (~45 sec): Be honest about what we'd redo. "These are all small adjustments. The core 4-collection / 8-index design held up well under actual implementation across all 6 queries."

Slide 12 - Lessons + Wrap

On the slide:

- 4 lessons:
  - `mongodb-memory-server` shrinks the design loop from minutes to seconds - default for prototyping
  - `explain('executionStats')` is the single most valuable design-validation tool
  - Multi-step orchestration in app code beats nested `$lookup` for clarity (Q3, Q4, Q6)
  - Schema design happens at index-design time, not collection-design time
- Repo + run instructions:
  - "Repo: github.com/abachu2005/cs498-airbnb-mongodb-prototype"
  - "Run it: `cd stage3-prototype && npm install && npm start` (or `npm run web` for the demo UI)"
  - "Evidence: `out/load_evidence.txt`, `out/q*_results.json`, `out/explain_q*.json`"
- "Questions?"

Speaker notes (~1 min): Two big takeaways. "First, don't start a Mongo project by setting up Atlas. `mongodb-memory-server` gives you a real Mongo with zero setup, and we used it to load the full InsideAirbnb datasets for four cities in under five minutes. Second, always commit `explain` output alongside your query code. Pseudocode in our Stage 2 was guessing about index usage; one line of `.explain()` per query in Stage 3 turned guessing into proof."

How this maps to the assignment scoring

- **Queries (20 pts):** All 6 implemented => the 4-of-6 minimum for a 3-person team is exceeded => +10 extra-credit eligible.
- **Data (10 pts):** Full InsideAirbnb datasets for all 4 cities loaded => "use of all or nearly all the data" target met.
- **Presentation (30 pts):** This deck covers all four required content points (data model on slide 3, 2+ query demos on slides 5-10, critique on slide 11, lessons on slide 12).
- **Individual report (40 pts):** Companion `STAGE_3_Report_MongoDB_Airbnb.pdf`.